

Intermediate (two-level logger) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/logging_practice.py` . Run it with `cd practice && python logging_practice.py` .

Used later by: the [Real-world wiring exercise](#) copies the `get_logger()` you write here into `practice/config_logging_wiring/logging_setup.py` , and the [Build Task](#) writes the same function again in `practice/build_task/logging_setup.py` . A copy, not an import — each of those folders stands alone. Keep the name and signature exactly as they are: `get_logger(name: str) -> logging.Logger` .

Basic Version

Approach 1 — the direct way

```
# practice/logging_practice.py
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

screen_handler = logging.StreamHandler()
screen_handler.setLevel(logging.INFO)
logger.addHandler(screen_handler)

file_handler = logging.FileHandler("app.log")
file_handler.setLevel(logging.DEBUG)
logger.addHandler(file_handler)

logger.debug("This only goes to the file.")
logger.info("This goes to both the screen and the file.")
logger.warning("This also goes to both.")
```

Expected output (printed to your terminal):

```
This goes to both the screen and the file.
```

```
This also goes to both.
```

The `debug` line is missing from the terminal on purpose — the screen handler's level is `INFO`, so it filters `DEBUG` out. It still reaches the file, because the file handler's level is `DEBUG`.

`app.log` after this runs (no formatter yet, so just the bare messages):

```
This only goes to the file.  
This goes to both the screen and the file.  
This also goes to both.
```

This version works correctly. It's missing type hints and a reusable function around the setup — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — wrapped in a typed function

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)

    return logger

logger = get_logger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both the screen and the file.")
logger.warning("This also goes to both.")
```

Expected output (terminal):

```
This goes to both the screen and the file.
This also goes to both.
```

`app.log` after this runs:

```
This only goes to the file.
This goes to both the screen and the file.
```

This also goes to both.

Approach 2 — same result, built from a list of handler specs

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    """Create a logger that prints INFO+ to the screen and DEBUG+ to a file."""
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    handler_specs = [
        (logging.StreamHandler(), logging.INFO),
        (logging.FileHandler("app.log"), logging.DEBUG),
    ]
    for handler, level in handler_specs:
        handler.setLevel(level)
        logger.addHandler(handler)

    return logger

logger = get_logger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both the screen and the file.")
logger.warning("This also goes to both.")
```

Expected output (terminal):

```
This goes to both the screen and the file.
This also goes to both.
```

app.log after this runs:

```
This only goes to the file.
This goes to both the screen and the file.
This also goes to both.
```

Difference from Basic: both Intermediate approaches move the setup into a typed function, `get_logger(name: str) -> logging.Logger`, so it can be reused for more than one logger

without copy-pasting the whole block. Approach 2 also adds a docstring and loops over a list of `(handler, level)` pairs instead of writing out `.setLevel()` / `.addHandler()` twice by hand — purely a style choice for avoiding repetition, not a behavior difference from Approach 1.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — two handlers directly, made safe to call twice

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    """Create (or reuse) a logger with a screen handler and a file handler.

    Safe to call more than once – if this logger already has handlers,
    they are not added again.
    """
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)

    return logger

logger = get_logger(__name__)
logger = get_logger(__name__) # called again – no duplicate handlers added

logger.info("This goes to both the screen and the file.")
```

Expected output (terminal):

This goes to both the screen and the file.

Printed once, not twice. Without the `if logger.handlers:` guard, the second `get_logger()` call would attach a second screen handler and a second file handler, and this line would print twice.

Approach 2 — with a shared formatter (what you'd actually ship)

A formatter makes your log lines useful later — timestamp, level, and module name, not just the bare message.

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    formatter = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s")

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    screen_handler.setFormatter(formatter)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)

    return logger

logger = get_logger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both.")
```

Expected output (terminal):

```
2024-01-15 10:23:01,123 [INFO] __main__: This goes to both.
```

The date and time will be whatever it actually is when you run it (down to the millisecond) — everything else in the line is fixed. The logger name shows as `__main__` because that's what

`__name__` equals when you run this file directly with `python script.py`.

Approach 3 — declarative setup with `logging.config.dictConfig()` (production style)

```
# practice/logging_practice.py
import logging
import logging.config

LOGGING_CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "standard": {
            "format": "%(asctime)s [%(levelname)s] %(name)s: %(message)s",
        },
    },
    "handlers": {
        "screen": {
            "class": "logging.StreamHandler",
            "level": "INFO",
            "formatter": "standard",
        },
        "file": {
            "class": "logging.FileHandler",
            "level": "DEBUG",
            "formatter": "standard",
            "filename": "app.log",
        },
    },
    "loggers": {
        "__main__": {
            "handlers": ["screen", "file"],
            "level": "DEBUG",
        },
    },
}

logging.config.dictConfig(LOGGING_CONFIG)
logger = logging.getLogger(__name__)

logger.debug("This only goes to the file.")
logger.info("This goes to both.")
```

Expected output (terminal):

```
2024-01-15 10:23:01,123 [INFO] __main__: This goes to both.
```

Same line as Approach 2 — `dictConfig` builds the exact same two handlers and formatter, just from one dictionary instead of several lines of Python. Note the `"loggers": {"__main__": {...}}` key only matches when this file is run directly; a real multi-module app usually configures the **root** logger instead (the key `"` in place of `"__main__"`), so every module's `logging.getLogger(__name__)` inherits the same handlers automatically.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate's `get_logger()` works the first time but breaks if it's ever called twice — every handler gets duplicated, and every log line prints multiple times. Approach 1 fixes exactly that, with a 2-line guard. Approach 2 adds a shared formatter, so log lines carry a timestamp and level instead of being bare text — the difference between a log file you can actually debug from and one you can't. Approach 3 does the same job as Approach 2 but describes it *declaratively*, as one dictionary (which in a real app is usually loaded from a JSON or YAML config file) instead of *imperatively* building handler objects in Python — the advantage being that ops or another developer can change log levels or add a handler by editing config, without touching code.

Which one should you actually write? For an exercise or a small script, Intermediate Approach 1 (a typed `get_logger` function, no extra machinery) is completely correct and what most people should default to. Advanced Approach 1's re-entry guard is worth adding the moment this function might run more than once — which, in a real app with multiple modules importing each other, is more common than it sounds. Advanced Approach 2's formatter is worth it the moment anyone other than you will read the log file. Advanced Approach 3's `dictConfig` is worth it specifically once logging configuration needs to change without a code change (different levels in production vs. development, for example) — don't reach for it by default, it's solving a problem a small script usually doesn't have yet.